Exploring Scalable Linear Solvers with *hypre* and *hypredrive*

90-minute lecture and local hands-on session

Victor A. P. Magri
Center for Applied Scientific Computing, LLNL





Prepared by LLNL under Contract DE-AC52-07NA27344.

Our 90-minute session

Elapsed	Topic	Minutes
00–05	Introduction and local readiness	5
05–20	Sparse systems and Krylov methods	15
20–30	**Lab 1: convergence and restart**	**10**
30–45	Algebraic multigrid and hierarchy cost	15
45–55	**Lab 2: problem growth and AMG tuning**	**10**
55–58	Solver portfolio and MGR	3
58–63	**Lab 3: Darcy flow with MGR**	**5**
63–68	Matrix input and controlled experiments	5
68–83	**Lab 4: a real matrix from file**	**15**
83–85	Workflow and thank you	2
85–90	Questions and recovery time	5

By the end: choose a valid solver pair and compare its cost on your matrix.

Readiness check: one small solve on your laptop

CPU only, one thread per rank. Labs 1, 2 and 4 use one rank; Lab 3 uses eight.

```
# Before class, from the repository root
bash lab/setup.sh
source lab/activate.sh

# Quick readiness check
"$LAP" -ns 1 -n 12 12 12 \
  -i "$LAB/configs/pcg-amg.yml" -v 1
```

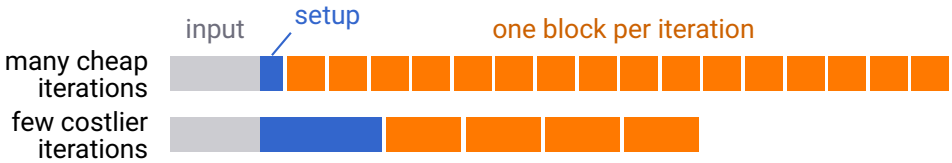
Success: one statistics row with a relative residual below 10^{-8}.

Not ready? Pair with a neighbor or read `lab/reference/linux-x86_64/`.

Prework and troubleshooting: `lab/README.md`. Installation runs before the 90-minute session.

The cost of a sparse solve has three parts

$$T_{\text{total}} = \underbrace{T_{\text{input}}}_{\text{read } A, b} + \underbrace{T_{\text{setup}}}_{\text{build } B \approx A^{-1}} + \underbrace{n_{\text{iter}}}_{\text{convergence}} \times \underbrace{T_{\text{iter}}}_{\text{SpMV + apply } B \text{ + dots}}$$

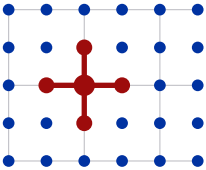


Schematic, not to scale. Lab 1 and Lab 4 measure both shapes.

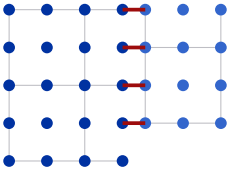
- One SpMV costs $O(\text{nnz}(A))$. Direct factorization adds fill and memory.
- Krylov storage stays modest, but n_{iter} can grow as the mesh is refined.

Source: Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., 2003, Chapters 6 and 9.

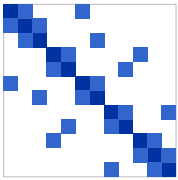
What the application tells hypre shapes the solver



Struct
Grid + stencil
PFMG, SMG



SStruct
Parts + connections
SSAMG or ParCSR methods



IJ / ParCSR (this class)
Matrix rows and entries
BoomerAMG, ILU, FSAI, MGR

Grids, field labels, and discrete operators enable methods a bare matrix cannot.

Source: hypre user manual, conceptual interfaces. AMS/ADS also require auxiliary discretization data.

A small residual need not mean a small error

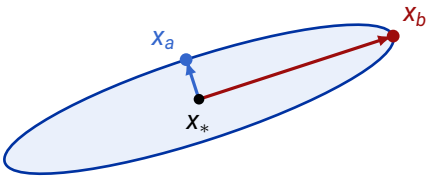
The solver measures the residual:

$$r_k = b - Ax_k, \qquad \rho_k = \frac{\|r_k\|_2}{\|b\|_2} \leq 10^{-8}$$

The application needs the error $e_k = x_* - x_k = A^{-1} r_k$:

$$\frac{\|e_k\|_2}{\|x_*\|_2} \leq \kappa_2(A)\, \rho_k$$

An iteration cap is not a convergence certificate.



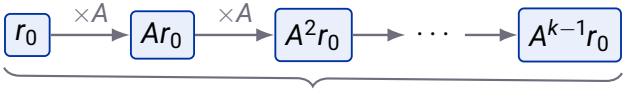
Same $\|b - Ax\|_2$ on the whole ellipse, yet $\|e_b\| = \kappa_2(A)\, \|e_a\|$

2-D SPD sketch, $\kappa_2(A) \approx 5$.

Compare runs at the same residual and tolerance. Lab 4 also measures the error.

Source: Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., 2003, Chapters 6 and 9.

A Krylov method grows a search space one SpMV at a time



$$\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, \ldots, A^{k-1}r_0\}, \quad x_k \in x_0 + \mathcal{K}_k$$

The method decides which x_k

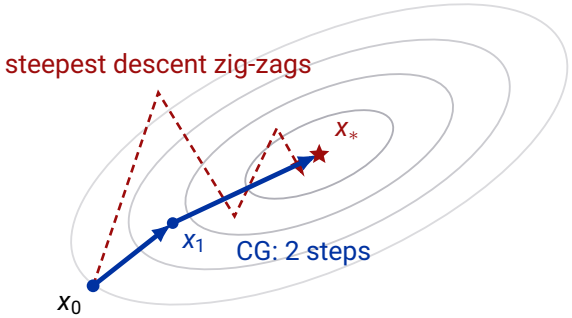
- CG: smallest error in the A-norm
- GMRES: smallest residual in the 2-norm

The preconditioner changes the operator

- The space is built from BA or AB, with $B \approx A^{-1}$
- A good B makes a small space sufficient

Source: Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., 2003, Chapters 6 and 9.

CG uses conjugate directions on SPD systems



For symmetric positive definite (SPD) A, x_* minimizes ϕ. CG picks

$$x_k = \arg \min_{x \in x_0 + \mathcal{K}_k} \|x_* - x\|_A.$$

A-conjugate directions keep earlier progress. A short recurrence keeps storage fixed.
PCG also needs a fixed SPD preconditioner.

In exact arithmetic, CG finishes this 2-D problem in at most two steps.

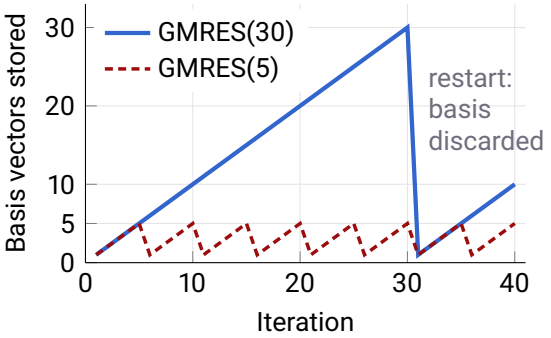
CG energy schematic. Theory: Saad, Chapters 6 and 9.

GMRES restart trades memory for retained information

For a general nonsingular A, GMRES minimizes the residual over its space:

$$x_k = \arg \min_{x \in x_0 + \mathcal{K}_k} \|b - Ax\|_2.$$

Step j adds a basis vector, orthogonalized against all j stored ones.
GMRES(m) discards the basis every m steps.

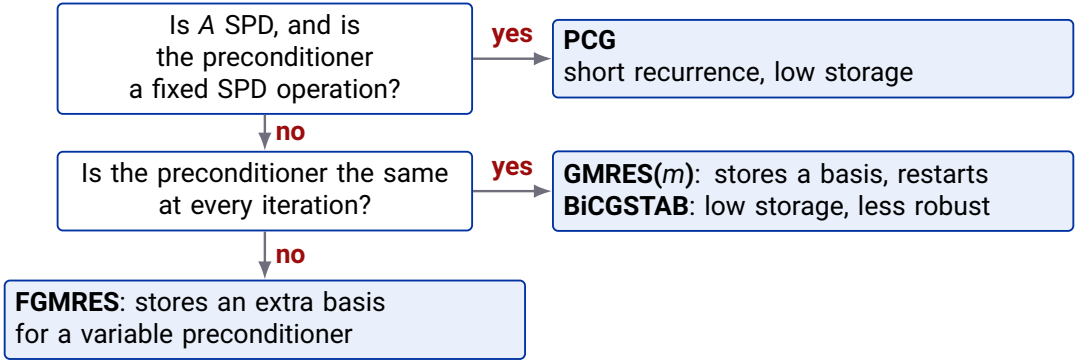


Storage pattern by definition, not a convergence history.

Lab 1 holds the preconditioner fixed and compares $m = 30$ with $m = 5$.

Unpreconditioned formulation shown. The minimized norm depends on the preconditioning convention. Saad, Chapters 6 and 9.

The matrix and preconditioner select the Krylov method



Upwind convection gives a nonsymmetric matrix. SPD diffusion permits PCG.

General matrices here mean nonsingular systems. Breakdowns remain possible. Saad, Chapters 6 and 9.

Preconditioning changes convergence and cost

eigenvalues of A: spread out



eigenvalues of BA: clustered



Schematic spectra.

For SPD A and B, the classical PCG bound grows like $\sqrt{\kappa(BA)}$.

$B \approx A^{-1}$	Form	Pairing in this lab
Jacobi	D^{-1}	PCG on SPD diffusion
FSAI	$G^T G$	PCG on SPD matrices
ILU	$(LU)^{-1}$	GMRES, since ILU need not be SPD
AMG	V-cycle	PCG with an SPD cycle

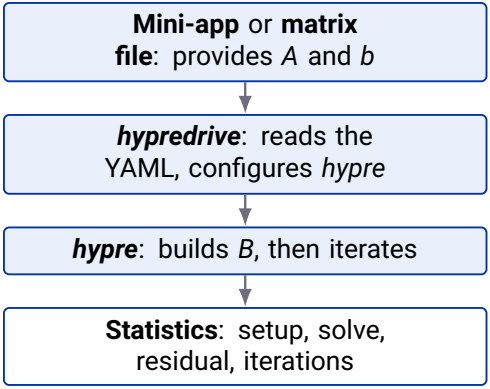
A stronger B costs more to build and apply. Fewer iterations can still be slower.

FSAI: factorized sparse approximate inverse. ILU: incomplete LU. Sources: hypre solver manuals and Saad, Chapters 9 and 10.

A YAML file states the solver policy

```yaml
solver:
  pcg:
    max_iter: 1000
    relative_tol: 1.0e-8
preconditioner: fsai
```

`solver` and `preconditioner` are separate choices. Edit the file; do not rebuild.



Source: hypredrive, pinned lab revision in `lab/versions.env`.

hypredrive: the same solve from six languages

Explicit lifecycle

C HYPREDRV.h

```
HYPREDRV_Create(comm, &hd);
HYPREDRV_InputArgsParse(argc, argv, hd);
HYPREDRV_LinearSolverApply(hd); // after Setup
```

C++ hypredrive.hpp

```
hypredrive::driver drv(comm);
drv.parse_yaml(yaml);
drv.solve();  // after set_matrix_from_csr(...)
```

Fortran use hypredrive

```
ierr = HYPREDRV_Create(comm, drv)
ierr = HYPREDRV_InputArgsParseYaml(drv, yaml)
ierr = HYPREDRV_LinearSolverApply(drv)
```

One high-level call

Python import hypredrive as hd

```
opts = hd.configure(solver="pcg",
    preconditioner="amg")
res = hd.solve(A, b, options=opts)
```

MATLAB / Octave

```
opts = hypredrive_options('solver', 'pcg', ...
    'preconditioner', 'amg');
[x, info] = hypredrive_solve(A, b, opts);
```

Julia using HypreDrive

```
opts = hypredrive_options(solver=:pcg,
    preconditioner=:amg)
x, info = hypredrive_solve(A, b; options=opts)
```

Today's lab uses C. Abridged; matrix/RHS setup omitted. Only C builds by default; enable others with `-DHYPREDRV_ENABLE_<LANG>=ON`. Checked against pinned `interfaces/`.

Lab 1: convergence and restart

20:00–30:00 10 minutes

```
bash lab/run.sh 1
```

1. First solve: 24^3 diffusion, PCG + Jacobi.
2. Same FSAI preconditioner: PCG, GMRES(30), BiCGSTAB at 24^3 and 48^3.
3. At 48^3: GMRES(30) versus GMRES(5).

Record for every run: converged below 10^{-8}? iterations, setup and solve ms.

Checkpoints 25:00 one converged solve · 29:00 one conclusion about restart

Worksheet and prompts: `lab/README.md`. Reference logs: `lab/reference/macos-arm64-m5/`.

The statistics row separates the costs

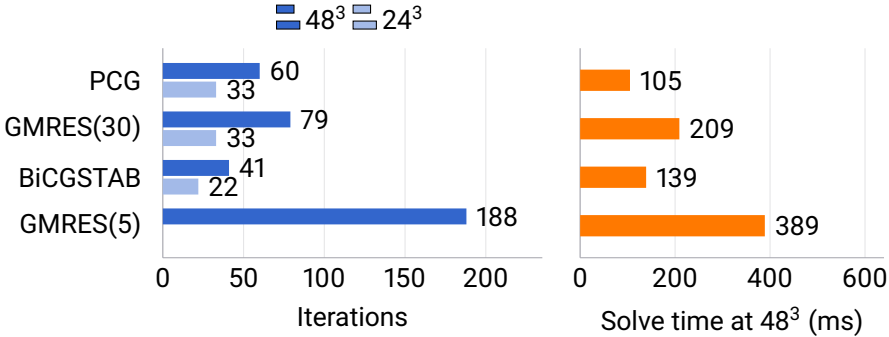
LS build	Setup	Solve	Relative residual	Iterations
	milliseconds			
1.20	0.01	7.70	8.09×10^{-9}	81

LS build	$\approx T_{\text{input}}$	the mini-app constructs A and b
Setup	$= T_{\text{setup}}$	builds the preconditioner and solver state
Solve	$= n_{\text{iter}} T_{\text{iter}}$	iterates to the tolerance

Today, total solver time means setup + solve. Input is reported separately.

Recorded M5 CPU run, hypredrive v0.4.0 / hypre v3.2.1, 1 rank, 1 thread. `lab/reference/macos-arm64-m5/`. Laptop times will differ.

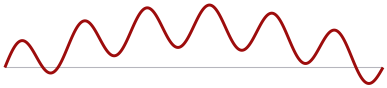
Lab 1 discussion: iterations are not equal units of work



Every 48^3 run has the same FSAI setup, 234–238 ms. Each BiCGSTAB iteration applies *A* and *B* twice. **Discuss:** what did GMRES(5) lose?

Recorded M5 CPU run, hypredrive v0.4.0 / hypre v3.2.1, 1 rank, 1 thread. `lab/reference/macos-arm64-m5/`. Laptop times will differ.

Relaxation removes rough error; a coarse grid handles smooth error



error before relaxation: rough + smooth

a few sweeps



after: the smooth part remains



fine grid h: 17 points, looks smooth

restrict

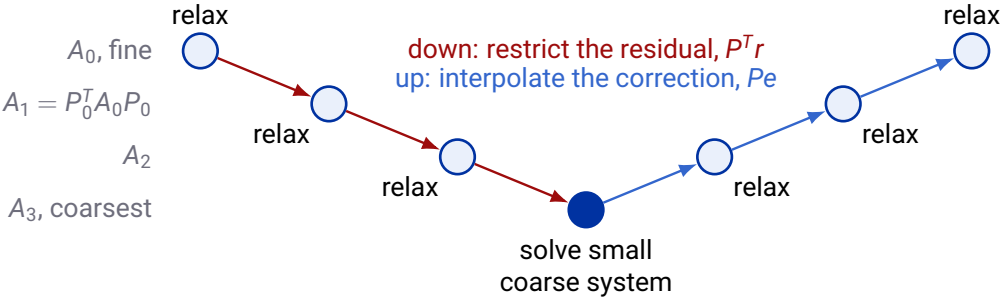


coarse grid $2h$: 9 points, rougher

On the coarse grid the same mode has fewer points per wavelength, so relaxation is effective again, at a fraction of the cost.

Schematic of the standard diffusion model problem. Not measured convergence data.

A V-cycle visits every level once on the way down and up

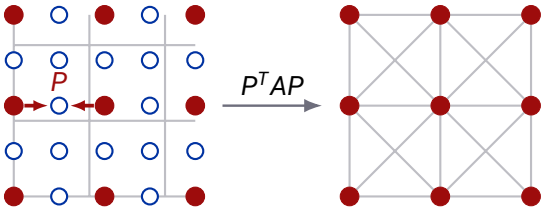


Work per cycle is proportional to $\sum_\ell \text{nnz}(A_\ell)$. With a bounded sum and a bounded number of cycles, total work is $O(N)$.

Both bounds depend on the operator and the hierarchy.

Galerkin coarse operators with restriction $R = P^T$, as used for SPD problems. Source: hypre BoomerAMG user manual.

BoomerAMG builds the hierarchy from matrix entries



2-D five-point example: A_c gains diagonal couplings.

Setup, once per matrix

C points from strong couplings

↓

interpolation P, restriction P^T

↓

coarse operator $A_c = P^TAP$

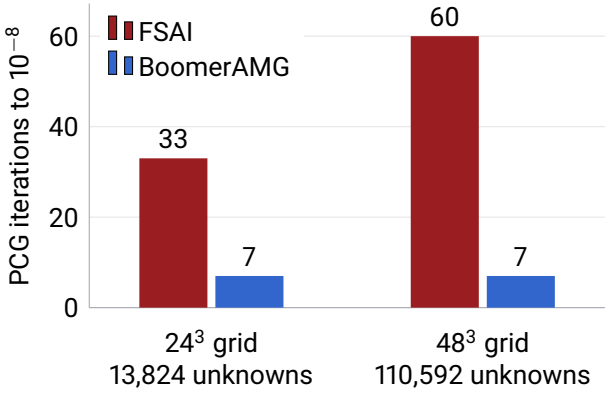
repeat

Apply: one V-cycle per Krylov iteration

No mesh is needed: the matrix graph plays the role of the grid.

Source: hypre BoomerAMG user manual.

AMG iterations stayed flat as the problem grew



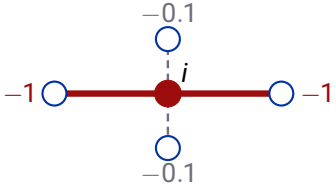
Same 7-point diffusion operator, PCG, tolerance 10^{-8}.
$8\times$ the unknowns:
FSAI $33 \rightarrow 60$ iterations
AMG $7 \rightarrow 7$ iterations
The hierarchy removes the smooth error that FSAI leaves.

Two sizes illustrate a trend. They do not prove scalability.

Recorded M5 CPU run, hypredrive v0.4.0 / hypre v3.2.1, 1 rank, 1 thread. `lab/reference/macos-arm64-m5/`. Laptop times will differ.

Strong couplings decide coarse points and interpolation

Which neighbors matter?

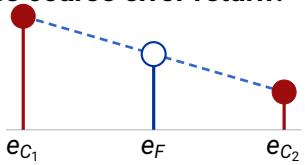


j strong for $i \iff -a_{ij} \geq \theta \max_{k \neq i}(-a_{ik})$

With $\theta = 0.25$, only the -1 couplings pass.

Source: hypre BoomerAMG user manual.

How does coarse error return?



ideal (dense): $e_F = -A_{FF}^{-1} A_{FC}\, e_C$

BoomerAMG keeps only strong C neighbors. Richer P costs setup and memory.

Smoothers trade local strength for parallelism

Smoother	Main consideration
Hybrid Gauss–Seidel	Sequential local sweeps: a strong CPU smoother
Jacobi, ℓ_1-Jacobi	Independent row updates: more parallelism
Chebyshev	Matrix–vector products replace triangular solves
ILU, FSAI	Stronger local correction can justify more setup

PCG needs an SPD preconditioner. A symmetric cycle, such as matching down and up sweeps, is part of that contract.

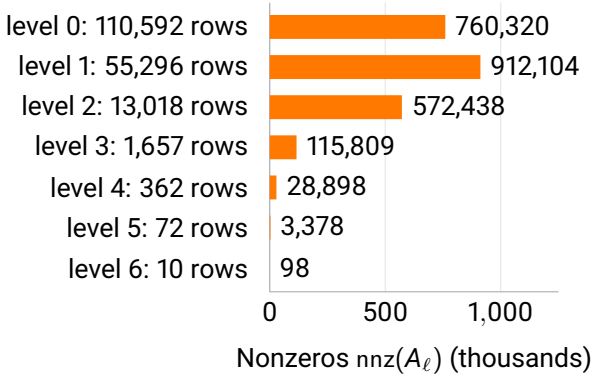
Source: hypre BoomerAMG user manual.

Coarse levels can hold more nonzeros than the fine level

$$C_{\text{grid}} = \frac{\sum_\ell n_\ell}{n_0} = 1.64$$

$$C_{\text{op}} = \frac{\sum_\ell \text{nnz}(A_\ell)}{\text{nnz}(A_0)} = 3.15$$

Level 1 has half the rows of level 0 and 1.2 times its nonzeros.
C_{op} tracks memory and V-cycle cost.



Default BoomerAMG hierarchy for the Lab 2 48^3 problem, 0 aggressive levels, from the recorded printout `amg-agg0.txt`. Source: hypre BoomerAMG user manual.

One setup can serve many right-hand sides

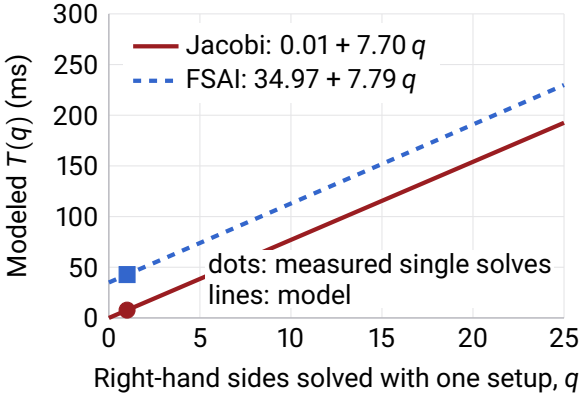
For q right-hand sides with the same matrix and policy:

$$T(q) = T_{\text{setup}} + q\,T_{\text{solve}}.$$

Lab 1 at 24^3, PCG:

- Jacobi: a small setup
- FSAI: a larger setup

Jacobi has lower setup and solve cost in this run. Reusing setup preserves that ranking.



Model from one recorded run per policy (dots). Assumes an unchanged matrix, reused setup, and comparable solve cost per right-hand side. Not a measured repeated-RHS benchmark. Input excluded.

Lab 2: problem growth with AMG

45:00–55:00 10 minutes

```
bash lab/run.sh 2
```

1. PCG + AMG at 24^3 and 48^3. Compare with Lab 1's FSAI.
2. At 48^3, use 0, 1, and 2 aggressive-coarsening levels.

Record for every run: operator complexity, iterations, setup and solve ms.

Decide: which hierarchy has the lowest setup + solve on your laptop?

Checkpoints 50:00 both sizes converged · 54:00 one tuning conclusion

Compare trends, not wall times between students. Worksheet: `lab/README.md`.

One baseline policy, one controlled change

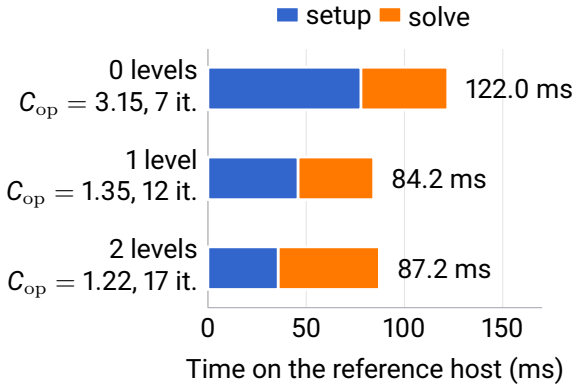
The runner changes a single parameter between runs:

```
"$LAP" -ns 1 -n 48 48 48 \
  -i "$LAB/configs/pcg-amg.yml" -v 1 -a \
  --preconditioner:amg:print_level 1 \
  --preconditioner:amg:aggressive:num_levels 1
```

- Options after `-a` override the YAML policy.
- `print_level 1` prints the hierarchy. Find the line `operator =`.
- The matrix, tolerance, and one-rank layout stay fixed.

Source: pinned hypredrive mini-app CLI and `lab/run.sh`.

Lab 2 discussion: the fewest iterations did not win



Aggressive coarsening lowered C_{op} and setup, but added iterations.
Discuss: is a 3.0 ms gap trustworthy from one run? Which wins for 20 right-hand sides?

Recorded M5 CPU run, hypredrive v0.4.0 / hypre v3.2.1, 1 rank, 1 thread. `lab/reference/macos-arm64-m5/`. Laptop times will differ.

Application structure points to a solver family

Problem information	Candidate	What the application supplies
Structured diffusion	PFMG / SMG	Grid and stencil
Structured parts	SSAMG	Parts and their connections
General elliptic, scalar	BoomerAMG	The matrix
Coupled fields	MGR	Field labels and a reduction order
Edge / face elements	AMS / ADS	Discrete gradient / curl, coordinates
Transport	AIR, ILU, Schwarz	Operator-dependent tuning

The application knows structure that a sparse graph alone does not reveal.

Source: hypre manuals for Struct, SStruct and ParCSR solvers. AIR: approximate ideal restriction. MGR: multigrid reduction.

MGR: multigrid reduction for a mixed Darcy system

Mixed Darcy flow

$K^{-1}\mathbf{u} + \nabla p = 0, \qquad \nabla \cdot \mathbf{u} = q$

$$\begin{pmatrix} M & B^T \\ B & 0 \end{pmatrix} \begin{pmatrix} \mathbf{u} \\ p \end{pmatrix} = \begin{pmatrix} f \\ g \end{pmatrix}$$

Flux **u**: F Pressure p: C

Approximate flux elimination leaves a pressure problem:

$$-S \approx B\widetilde{M}^{-1}B^T$$

AMG solves pressure C.

MGR = multigrid reduction preconditioner

1. Use field labels: flux is F; pressure is C.
2. Relax and approximately eliminate F, forming a reduced C problem.
3. Apply AMG to the pressure correction and return it to the full system.

MGR uses field labels to expose the pressure problem for AMG.

Schur reduction: hypre MGR manual. SPE10 field images from hypredrive documentation are in backup.

Lab 3: MGR on the SPE10 Darcy problem

58:00–59:00 5-minute lab; work through 62:00

Original SPE10 case 2a, sampled from $60 \times 220 \times 85$ to $30 \times 110 \times 85$ cells (about 1.14M unknowns). Run on eight ranks with a $2 \times 2 \times 2$ partition:

```
bash lab/run.sh 3
```

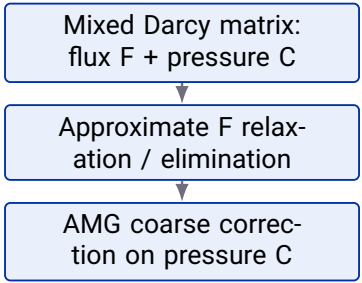
Only the pressure AMG cycle count changes: one versus two. `f_dofs: [1]` labels flux as F; pressure is the retained C field. **Record** convergence, GMRES iterations, setup, and solve:

Policy	AMG cycles	GMRES iters	Residual	Setup + solve (ms)
Default	1			
Stronger pressure solve	2			

Discuss: does the added coarse-solve work reduce Krylov work enough?

Lab 3: a physics-informed field split

- In Lab 3, `f_dofs: [1]` marks flux rows as F.
- Pressure rows are C and remain in the reduced system.
- The coarse AMG settings trade setup and application work against GMRES iterations.



Keep the field labels with the matrix: they tell MGR how to reduce it.

Compare the two raw runs in `$WORK/out/darcy-*.txt`; check convergence before comparing cost.

A Matrix Market file replaces application assembly

```
%%MatrixMarket matrix coordinate real symmetric
% name: Schmid/thermal1
82654 82654 328556
5 5 2.218274982935107
22859 5 -.1676377709699697
```

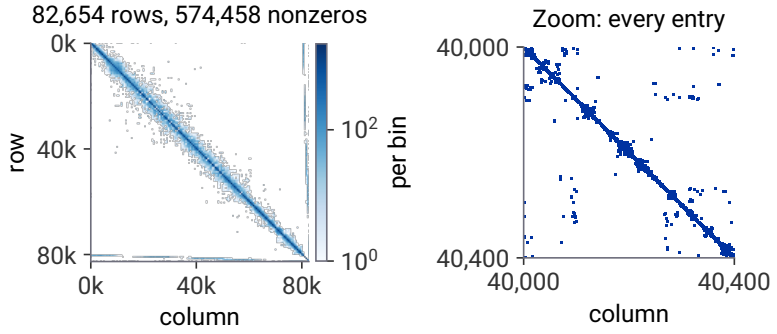
format, values, symmetry
comments carry provenance
rows, columns, stored entries
row i, column j, value a_{ij}
lower triangle only; $a_{ji} = a_{ij}$

```
linear_system:
  type: mtx
  matrix_filename: data/thermal1.mtx
  rhs_mode: randsol
  init_guess_mode: zeros
```

`randsol` fixes a reference solution x_{ref} and forms $b = Ax_{\mathrm{ref}}$.
Every policy then solves the same system from $x_0 = 0$. Because x_{ref} is known, the error $\|x - x_{\mathrm{ref}}\|$ is measurable.

First lines of the bundled `thermal1.mtx`; separator comments omitted. Excerpt from `lab/data/thermal1.mtx.gz`. Source: pinned hypredrive Matrix Market reader and input structure manual. This import path uses one MPI rank.

Schmid/thermal1: a real SPD finite-element matrix



Rows and columns in file order. The file has no coordinates, so no mesh is shown. Steady thermal problem, unstructured finite elements. SuiteSparse Matrix Collection, Schmid/thermal1, ID 1402, D. Schmid, University of Oslo. Plot: `scripts/thermal1-sparsity.py` from the bundled file.

Lab 4: solve a matrix from file

68:00–83:00 15 minutes

```
bash lab/run.sh 4
```

Policy file	Krylov method	Preconditioner
`file-amg.yml`	PCG	BoomerAMG
`file-fsai.yml`	PCG	FSAI
`file-ilu.yml`	GMRES(50)	ILU

Record: residual, error, iterations, setup and solve ms. **Write:** one recommendation, with evidence.

Checkpoints 73:00 file imported, one policy converged · 78:00 all three recorded

The file experiment holds the input fixed

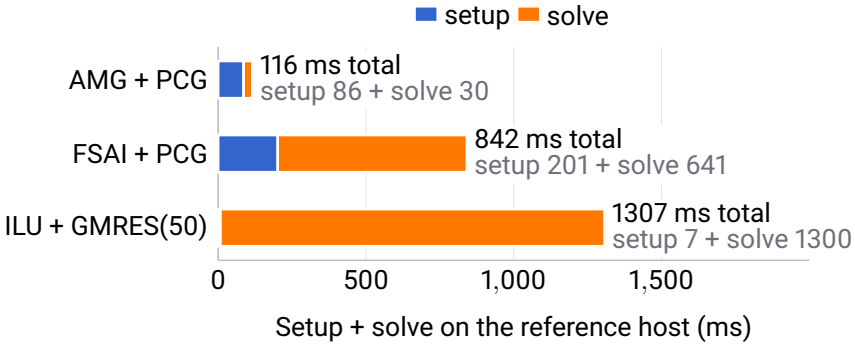
For each policy the runner executes, from the workspace that holds `data/`:

```
( cd "$WORK" && "$CLI" "$LAB/configs/file-amg.yml" )
```

- Check convergence before ranking any run.
- Compare setup + solve. Read/import time stays separate.
- Error = `L2 norm of error` / `L2 norm of ref. solution`.
- Repeat a timing before trusting a small difference.

Generic ILU uses GMRES to respect the Krylov contract. Prompts and a blank results table: `lab/README.md`.

Lab 4 discussion: AMG had the lowest setup + solve



Iterations: AMG 8, FSAI 328, ILU 450. ILU: cheapest setup, costliest solve.

Discuss: could any number of reused right-hand sides favor ILU or FSAI here?

Recorded M5 CPU run, hypredrive v0.4.0 / hypre v3.2.1, 1 rank, 1 thread. `lab/reference/macos-arm64-m5/`. Laptop times will differ.

Lab 4 discussion: the same residual, different errors

Policy	Iterations	Relative residual $\|b - Ax\|/\|b\|$	Relative error $\|x - x_{\mathrm{ref}}\|/\|x_{\mathrm{ref}}\|$
AMG + PCG	8	8.7×10^{-9}	4.7×10^{-8}
FSAI + PCG	328	9.4×10^{-9}	3.8×10^{-6}
ILU + GMRES(50)	450	9.9×10^{-9}	4.7×10^{-4}

All three passed the 10^{-8} residual test. Their errors span four orders of magnitude.

The residual test does not control which error components remain. CG minimizes the *A*-norm error; GMRES minimizes the residual.

Set the tolerance from the accuracy your application needs.

Recorded M5 CPU run, hypredrive v0.4.0 / hypre v3.2.1, 1 rank, 1 thread. `lab/reference/macos-arm64-m5/`. Laptop times will differ.

A workflow for your own sparse system

1. State the matrix class and the accuracy the application needs.
2. Select a compatible Krylov method and preconditioner.
3. Measure residual, error where possible, setup, and solve on the same input.
4. Change one choice, then repeat the comparison.
5. Revisit setup reuse and parallel layout for the real workload.

Export your matrix to Matrix Market and repeat Lab 4 on it.

Thank you



Thanks to the hypre team and the SciDAC/FASTMath community.

Victor A. P. Magri
Center for Applied Scientific Computing, LLNL
paludettomag1@llnl.gov

Resources and questions

hypre source documentation
hypredrive source documentation
Local exercises, policies and offline input: `lab/`

Funding and LLNL auspices

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC. LLNL-PRES-2026854.

This work was supported by the U.S. Department of Energy Office of Science, Office of Advanced Scientific Computing Research (ASCR), Scientific Discovery through Advanced Computing (SciDAC) program, the FASTMath Institute, and by the Exascale Computing Project, a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration.

Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.

Build details, additional experiments, and larger-machine evidence

These slides are outside the planned 90-minute sequence.

Backup: local installation and pinned versions

```
# macOS: install the Command Line Tools, then prerequisites
xcode-select --install
brew install cmake open-mpi python

# From the workshop repository root, before class
bash lab/setup.sh --jobs 4
source lab/activate.sh
```

- hypredrive: `v0.4.0`
- hypre: `v3.2.1`, fetched by CMake
- CPU Release build, MPI enabled, OpenMP and accelerators disabled

Linux/WSL2 and existing MPI installations: `lab/README.md`. More build jobs are optional on the presenter's M5.

Backup: AMS uses the curl null space

$$\nabla \times \mu^{-1} \nabla \times E + \sigma E = f$$



$\nabla \times (\nabla \phi) = 0$: gradients form a large null space of the curl term. AMS corrects them with G and nodal AMG.

Same matrix	AMG	AMS
PCG iterations	108	16

```
bash lab/run.sh maxwell
```

Optional, after class.

Source: hypre AMS manual. Recorded local CPU run, 12^3 nodal grid, 4,752 edge unknowns, PCG, $\sigma = \mu^{-1} = 1$, 10^{-8} tolerance.

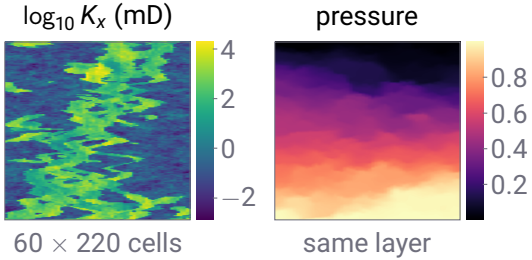
Backup: laptop experiments and larger machines

- Required solves use the CPU and one thread per rank; Lab 3 uses 8 ranks.
- Build jobs only shorten compilation. Runtime parallelism is a separate experiment.
- Small problems spend much of their time in setup; larger runs need matched problems and layouts.
- *hypre* accelerator builds use CUDA, HIP, or SYCL; this workshop uses the CPU.

A laptop timing cannot establish a GPU speedup or distributed scalability.

Source: hypre GPU documentation and hypredrive installation manual. Optional fixed-size MPI study: `bash lab/run.sh mpi`. CPU/A100 comparison: backup.

Backup: a heterogeneous Darcy application



SPE10 case 2a, layer 35: permeability input and computed pressure. Permeability varies by about seven decades. This motivates the mixed Darcy example; Lab 3 samples the original field on a $30 \times 110 \times 85$ grid.

Image from hypredrive documentation. Field labels and colorbar ticks re-typeset by `scripts/spe10-panels.py`.

Backup: nonsymmetric convection–diffusion

$$\frac{\partial c}{\partial t} + \boldsymbol{u} \cdot \nabla c - \kappa \Delta c = 0$$

Upwind transport makes $A \neq A^T$. The SPD CG assumptions no longer hold.

```
bash lab/run.sh transport
```

- $32 \times 8 \times 8$ grid, three time steps, GMRES + AMG.
- Read each statistics row as one time-step solve.
- A small observed residual would not establish that CG is valid for a nonsymmetric operator.

Source: hypredrive convection–diffusion mini-app. CPU-only optional extension.

Backup: a fixed-size MPI experiment

```
bash lab/run.sh mpi

# Representative two-rank command
mpiexec -n 2 "$LAP" -ns 1 -n 48 48 48 -P 2 1 1 \
  -i "$LAB/configs/pcg-amg.yml" -v 1
```

- Hold the global 48^3 problem fixed, use 1, 2, then 4 ranks.
- Match the processor grid to the number of MPI ranks.
- Compare setup and solve separately. Repeat timings.

This measures a small shared-machine case. It does not establish distributed scaling.

Backup: five SuiteSparse matrices to try

ID	Group / name	Rows	What to explore
1883	McRae/ecology2	999,999	Ecological diffusion
1403	Schmid/thermal2	1,228,045	Larger thermal model
2547	Janna/StocF-1465	1,465,137	Stochastic porous flow
2543	Janna/Fault_639	638,802	Fault contact mechanics
2542	Janna/Emilia_923	923,136	3-D geomechanics

After setup and activation, choose an ID:

```
python3 lab/suitesparse.py run 2547
```

All five are real SPD: compare PCG + AMG/FSAI and GMRES(50) + ILU.

Optional after class; the Janna cases need more memory than thermal1.

Collection IDs and verified input checksums: `lab/matrices.json`. Recorded M5 logs, exact policies and results: `lab/reference/macos-arm64-m5/suitesparse/`. Compare convergence and error before timings.

Backup: Polaris CPU/A100 comparison

Problem	Resource	Setup	Solve	Sum
		milliseconds		
128^3	32 CPU ranks	303	142	445
128^3	One A100	58	41	99
192^3	32 CPU ranks	1004	711	1715
192^3	One A100	165	120	285

Same global 7-point Poisson problem, PCG + AMG. Medians of warm solver lifecycles. Input/assembly and initial warm-up excluded.

Recorded 2026-07-24, AMD EPYC 7543P and A100 40 GB. Aggregate data and source revisions: `lab/reference/polaris-cpu-a100/`.

Backup: when setup reuse is worthwhile

Suppose policy 1 has more expensive setup but a cheaper application than policy 2. The crossover number of right-hand sides satisfies

$$q > \frac{T_{\mathrm{setup},1} - T_{\mathrm{setup},2}}{T_{\mathrm{solve},2} - T_{\mathrm{solve},1}}$$

when the denominator is positive.

- Use measurements at the same tolerance and execution layout.
- Keep the matrix and setup policy unchanged for exact reuse.
- If the matrix evolves, monitor convergence and choose a rebuild rule.

The cost model excludes input and assembly and assumes comparable solve costs across right-hand sides.

Backup: reading and source material

- Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003. Author's text.
- hypre user manual: BoomerAMG, MGR, AMS/ADS, structured solvers and GPU options.
- V. A. P. Magri, *Hypredrive: High-level interface for solving linear systems with hypre*, JOSS 9(98), 6654, 2024. doi:10.21105/joss.06654.
- T. A. Davis and Y. Hu, *The University of Florida Sparse Matrix Collection*, ACM TOMS 38(1), 2011. doi:10.1145/2049662.2049663.

Software and matrix provenance, template identity, exact lab revisions and raw measurements: `SOURCES.md` and `lab/reference/`.